Everything you need to know – Decoder's Blog

decoder.cloud/2025/04/24/from-ntlm-relay-to-kerberos-relay-everything-you-need-to-know

Decoder April 24, 2025

While I was reading Elad Shamir recent excellent post about NTLM relay attacks, I decided to contribute a companion piece that dives into the mechanics of Kerberos relays, offering an analysis and practical insights into how these attacks work and how they differ from NTLM based relays.

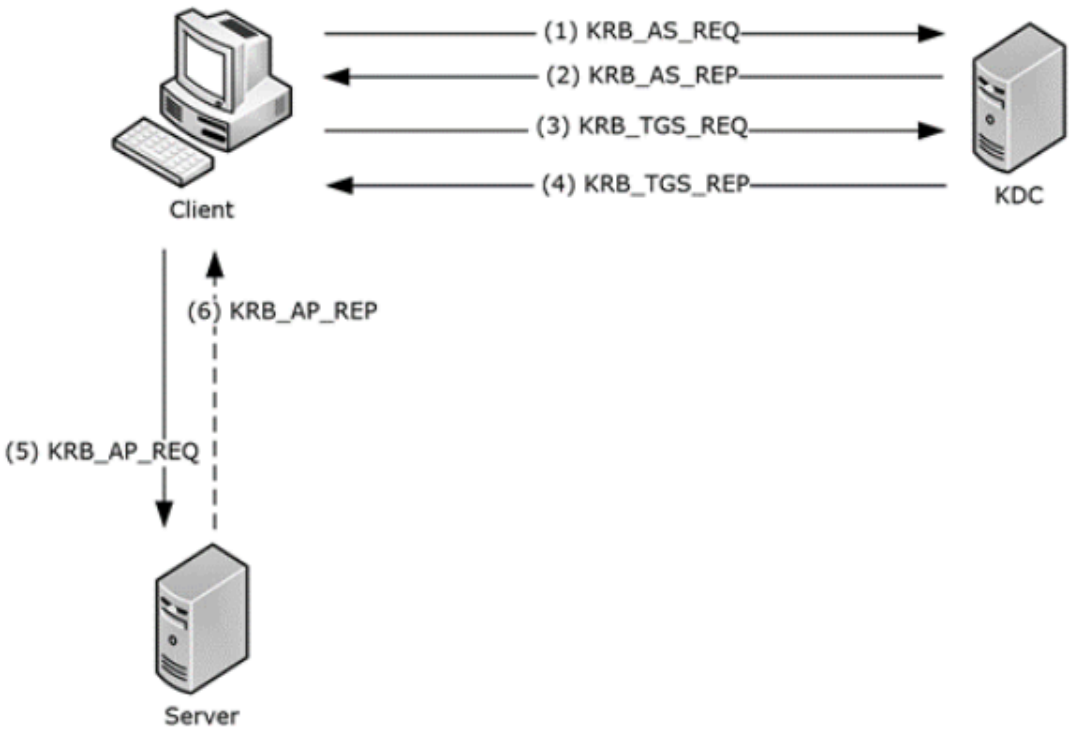
If you've been following my posts , tweets or checking out my GitHub, you probably know that lately I've been diving into Kerberos relay, trying to understand them better and demystify some concepts around them.

This post isn't meant to dive too deep into the technical details, but rather to give a general idea of how Kerberos relay attacks work, what's possible, what isn't, and where the limitations lie. My goal is to (hopefully) provide an easy-to-read, approachable overview that helps make sense of the bigger picture, explain how my dedicated KrbRelayEx tools work in practice, and show how they can be used to perform these kinds of attacks.

A Brief Recap of Kerberos Fundamentals

Kerberos is a network authentication protocol designed to securely verify the identities of users and services over an untrusted network. It uses symmetric key cryptography and a trusted Key Distribution Center (KDC) to enable secure communication without transmitting sensitive data like passwords.Its main components include:

- Authentication Service (**AS**): Verifies client credentials and issues Ticket-Granting Tickets (**TGT**s).
- Ticket-Granting Service (**TGS**): Provides service-specific tickets based on a valid TGT.
- Client-Server Model: The **AP-REQ** (Authentication Protocol Request) message is used **by a client to authenticate to a service** after obtaining a service ticket (TGS).



[Kerberos Network Authentication Service (V5) Synopsis]

This multi-step process ensures mutual trust between client and server while protecting sensitive information across the network.

What is kerberos relay

There hasn't been much documentation on this topic, especially compared to NTLM relay. CrowdStrike researchers gave a great [presentation] on Kerberos man-in-the-middle (MITM) attacks at DEFCON in 2021, which helped bring some attention. But the real *game changer* was an [article] by James Forshaw that went deep into the details of how Kerberos relay can be abused. I highly recommend reading it.

But as promised, I want to keep things simple in this post!

First of all let's understand better the mechanism.

Kerberos authentication requires the target service to be known using a Service Principal Name (SPN). This is a unique string that identifies the service, often in the format **CLASS/INSTANCE:PORT/NAME**. For example, *HTTP/webserver.domain.com* or *CIFS/fileserver.domain.com*

When a client wants to access a service, the Kerberos Ticket Granting Service (TGS) uses the Service Principal Name (SPN) to determine which account in Active Directory the

request is for. It then uses the corresponding encryption key,derived from that account's password (typically a machine account),to generate a service ticket. This ticket includes the user's identity and is tied to the original Ticket Granting Ticket (TGT).

The client sends this ticket to the target service as part of the **AP_REQ** message.

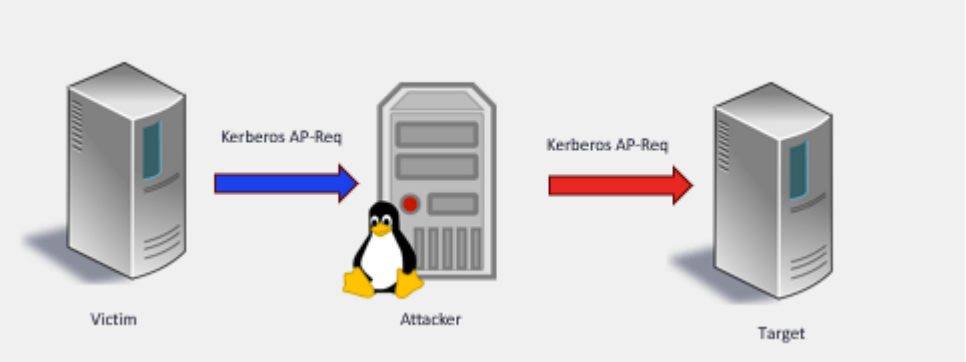
Because the encryption key is based on the account that owns the SPN, not the SPN string itself, tickets can often be relayed between different services (e.g., from HTTP to SMB) as long as both SPNs are registered to the same account. Unless the target service explicitly verifies the SPN in the ticket (which is rarely enforced), the authentication will succeed.

But, in contrast to NTLM, we cannot relay to another host, as the encryption key, based on the account's password, will change.

With this said, Kerberos relay attacks are conceptually far more straightforward than NTLM relays. Fundamentally, they involve intercepting the AP-REQ, the critical message sent by the client to a service (such as a server or application) to authenticate its identity and gain access.

This is how and AP-REQ looks like, we can easily identify the Service Principal Name:

If an attacker is able to intercept or coerce the AP-REQ request, they could use it to authenticate to the target server on behalf of the victim:



Before we delve into the technical details, let's first explore the limitation

Kerberos relaying shares the same limitation as NTLM relaying, namely:

- Encryption and integrity checks may prevent relaying protocols such as SMB, LDAP(S) , RPC, etc..
- Extended Protection for Authentication (EPA) can further restrict relaying for protocols such as HTTP and LDAPS

These limitations are well explained in ths excellent article regarding NTLM relay but alsow valid for Kerberos.

There is a third limitation, which isn't a real one, the **Mutual Authentication** request. Normally this can be safely ignored by the client except certain conditions, but more on this later.

Okay, that's the theory, but how does it work in practice?

At the core, all we need to do is extract the **Security Blob** , typically wrapped in GSS-API, sent by the client during the authentication process, and embed it into a new request targeting the desired service.

For example, we can insert that blob into an **SMB Session Setup Request** when relaying to an SMB service or, if the target service is HTTP(S), we simply Base64-encode the blob and include it in the Authorization: **Negotiate <Security Blob>** HTTP header.

As you might imagine, you need to manipulate raw packets over sockets, something that requires certain understanding of the underlying protocols.

For example, in the screenshot below we extract the **Security Blob** from an SMB Header:

```
public static byte[] ExtractSecurityBlob(byte[] sessionSetupRequest)
{
    // SMB2 Header is usually 64 bytes
    int smb2HeaderLength = 64;

    int securityBufferOffsetPosition = smb2HeaderLength + 12; // SecurityBufferOffset at byte 12 after header
    int securityBufferLengthPosition = smb2HeaderLength + 14; // SecurityBufferLength at byte 14 after header
    int securityBufferOffset = BitConverter.ToUInt16(sessionSetupRequest, securityBufferOffsetPosition);

    int securityBufferLength = BitConverter.ToUInt16(sessionSetupRequest, securityBufferLengthPosition);
    byte[] securityBlob = new byte[securityBufferLength];
    Array.Copy(sessionSetupRequest, securityBufferOffset, securityBlob, 0, securityBufferLength);

    return securityBlob;
}
```

In the next one, we embed the **Security Blob** in an HTTP header:

```
var AuthMessage = string.Format("Negotiate {0}", Convert.ToBase64String(SecurityBlob));

using (var message = new HttpRequestMessage(HttpMethod.Get, endpoint))
{
    message.Headers.Add("Authorization", AuthMessage);
    message.Headers.Add("Connection", "keep-alive");
    message.Headers.Add("User-Agent", "Mozilla/5.0 (Windows NT 6.1; WOW64; Trident/7.0; AS; rv:11.0) like Gecko");
    result = httpClient.SendAsync(message).Result;
}
Console.WriteLine("[*] HTTP Status Code:{0}", result.StatusCode);
Console.WriteLine("[*] HTTP Headers\r\n---");
Console.WriteLine(result.Headers);
Console.WriteLine("---");
```

Luckily, I didn't need to start from scratch, as Cube0x0 had already created an excellent framework for relaying DCOM over Kerberos.

Now that we have a better understanding of the low-level mechanisms, let's take a look at how we can actually implement this in practice.

Let's consider two scenarios:

1. Spoofing DNS names
2. Having control over the SPN

Spoofing dns host name resolution

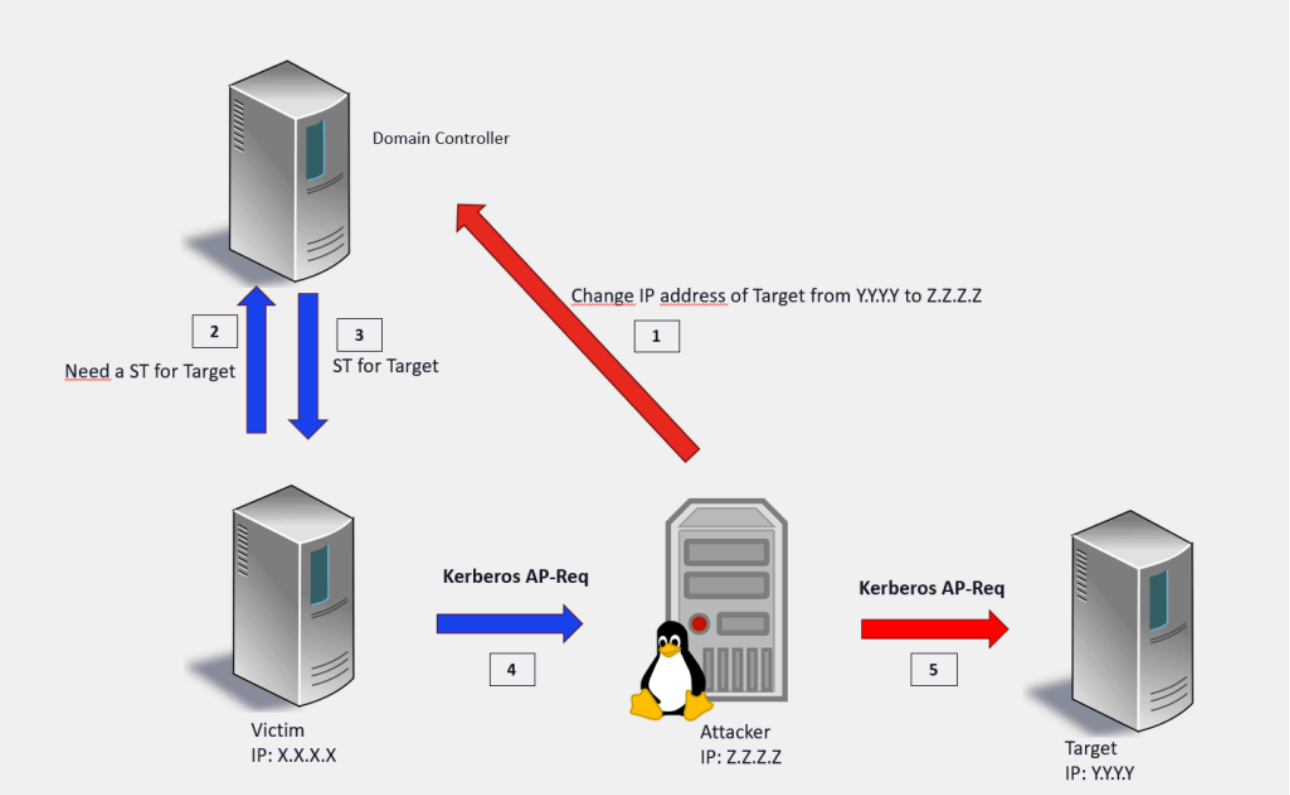
In this "man-in-the middle" scenario, an attacker is able to spoof hostname resolution, tricking the victim into believing it is communicating with the legitimate target

There are several techniques for spoofing host resolution, let's take a look at some common ones.

The attacker is a member of the **DnsAdmins** group. Membership in this group allows a user to modify any DNS records within the domain.

Unsecure DNS Updates are permitted, a dangerous misconfiguration that allows unauthenticated users with network access to perform DNS updates and potentially take over the domain.

The possible attack path is shown in the figure below:



Another method, for example, involves abusing weak permissions on the **HOSTS** file for hostname spoofing. By modifying entries in the HOSTS file on client machines, an attacker can redirect traffic destined for a specific hostname or FQDN to an arbitrary IP address.

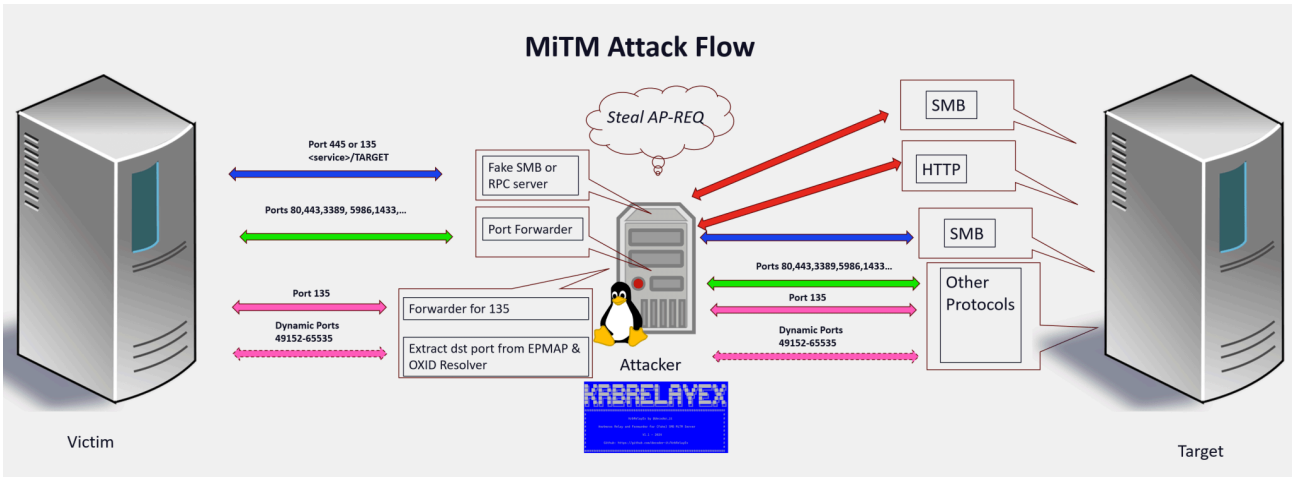
Once the client is redirected to our malicious host, we need to intercept the **AP-REQ** and forward the ticket to the desired target service. This is where my **KrbRelayEx** tools come into play.

I created two main tools:

- KrbRelayEx:
  - Implements a fake SMB server
  - Listens for incoming SMB session setup requests, extracts and forwards the AP-REQ to the target host, enabling access to SMB shares or HTTP ADCS (Active Directory Certificate Services) endpoints on behalf of the targeted identity.
  - Forwards the victim's requests dynamically and transparently to the real SMB destination so that the victim is unaware that their requests are being intercepted and relayed
  - Acts as a forwarder for the other protocols

- KrbRelayEx-RPC:
  - Implements a fake RPC/DCOM server
  - Listens for authenticated requests and extracts the AP-REQ tickets
  - Extracts dynamic port bindings from **EPMAPPER/OXID** resolutions
  - Relay the AP-REQ to access SMB shares or HTTP ADCS (Active Directory Certificate Services) on behalf of the victim
  - Forwards the victim's requests dynamically and transparently to the real destination RPC/DCOM application so the victim is unaware that their requests are being intercepted and relayed
  - Acts as a forwarder for the other protocols

The diagram below illustrates the full logical flow implemented by my tools:

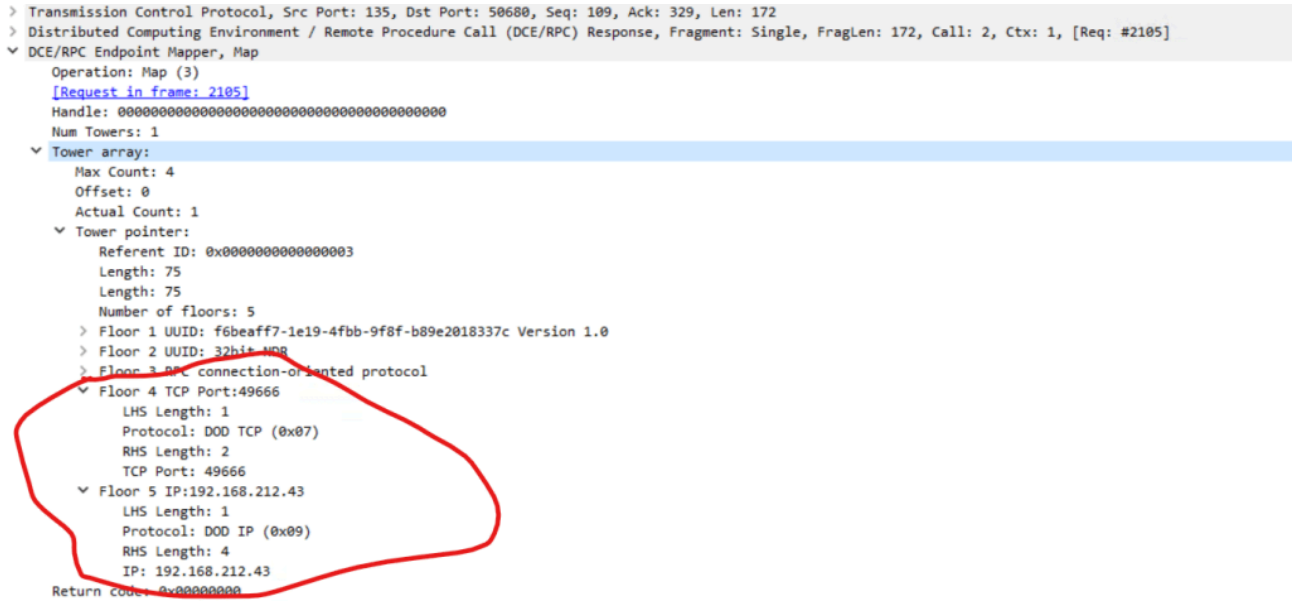


KrbRelayEx listens for incoming connections on SMB port 445 or RPC port 135, analyzes the raw input packets, and based on the relay rules, uses the extracted AP-REQ to authenticate to the target service on behalf of the victim. It then closes the original connection, causing the victim to retry ,and this time the connection succeeds, as KrbRelayEx is now in forward mode.All other ports specified via command line are simply forwarded to the destination host to ensure service availabilty and avoid disruption.

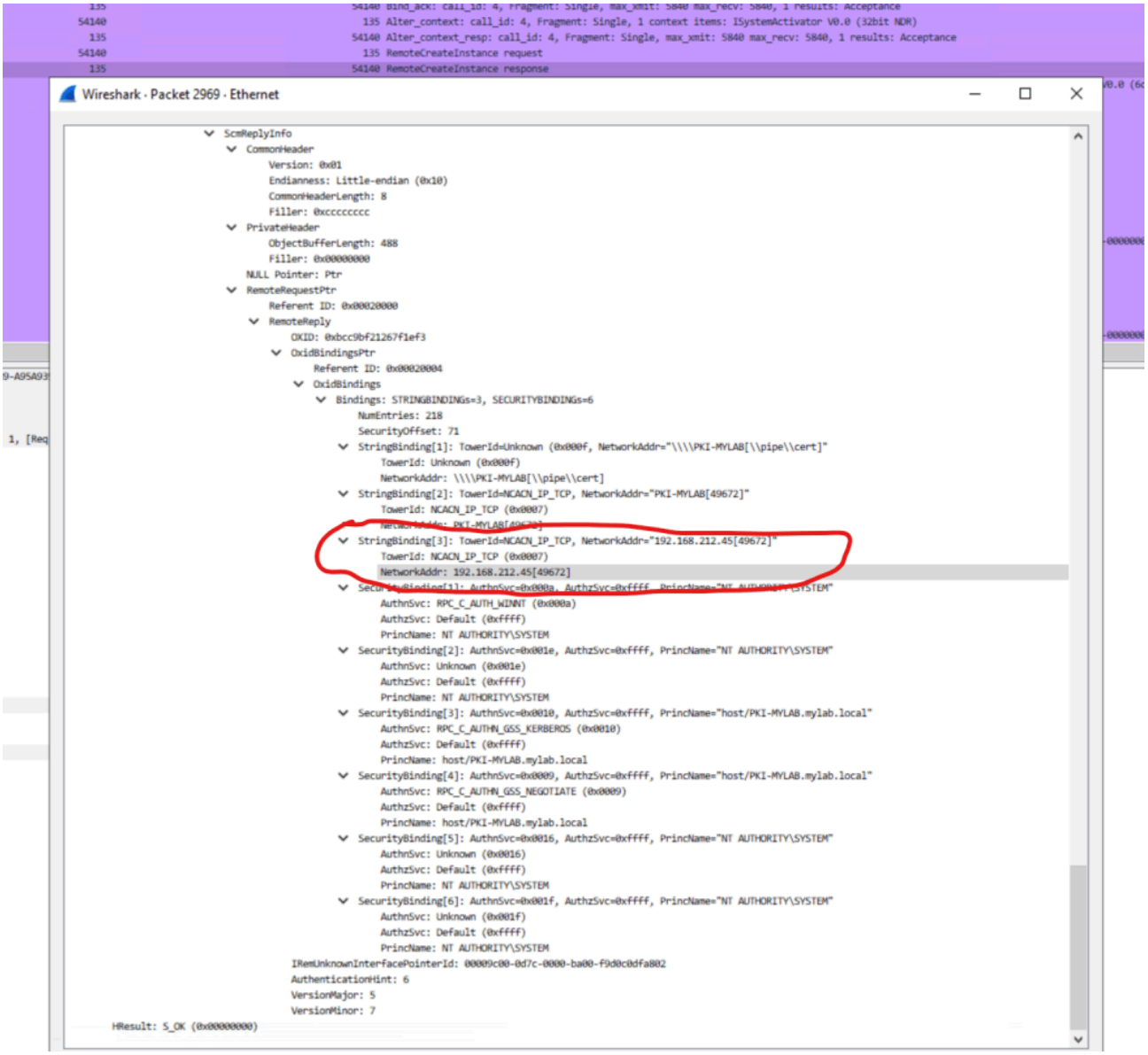
Side note on RPC/DCOM: It all starts with port 135. This port is particularly important for these protocols because it's used by services that handle resolution tasks, similar to a DNS for RPC and DCOM communications.

There are two types of "resolvers":

> The **EPMAPPER** service which acts as a directory service for **RPC communication**. It maps service endpoints to dynamically assigned ports so clients can locate and connect to RPC services. The service returns the remote port where the desidered service is running, along with the protocol sequence (nacn_tcp)



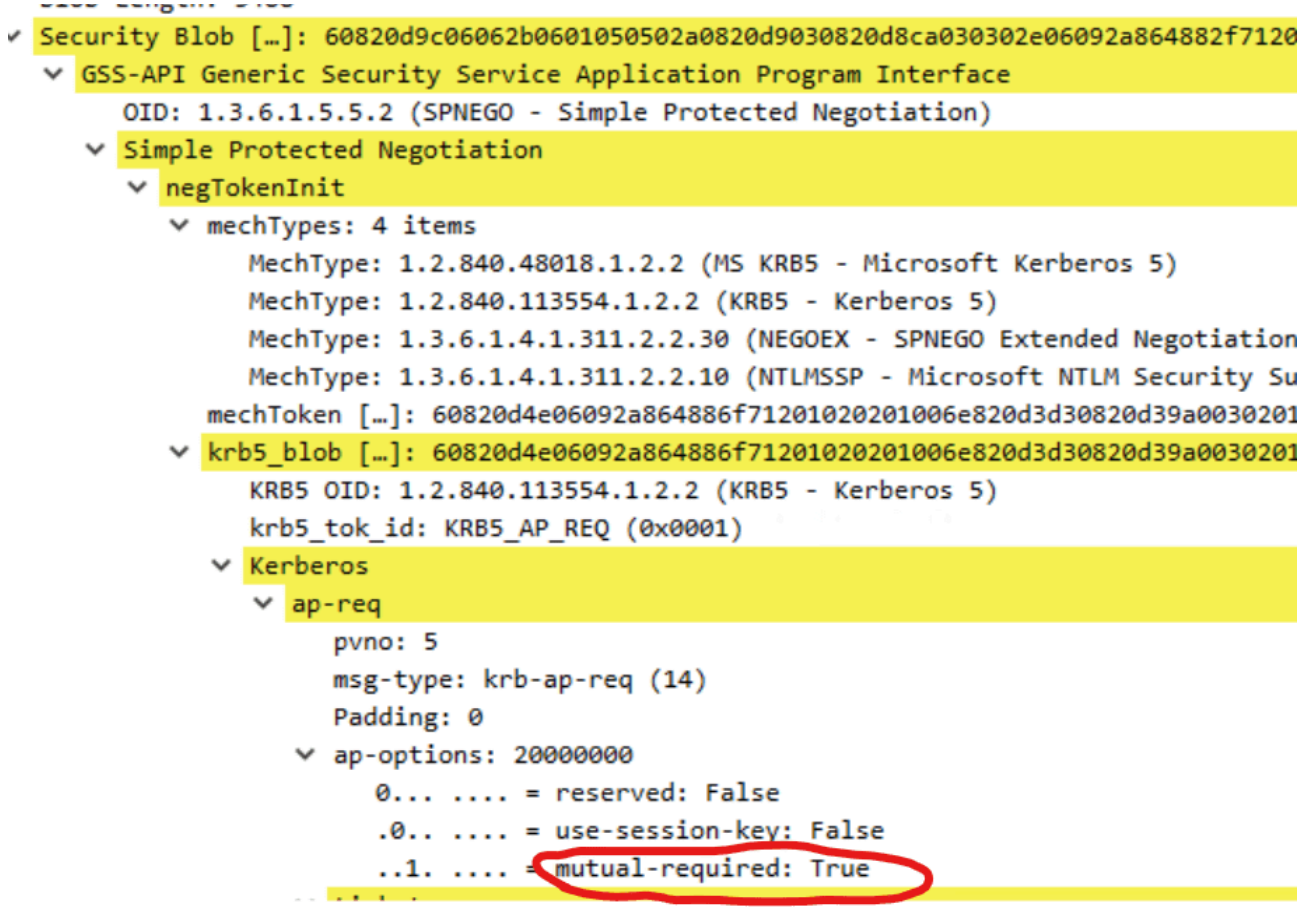
The **OXID Resolver** resolves Object Exporter Identifiers (OXIDs) into their corresponding network addresses (IP and port), enabling clients to locate and communicate with specific DCOM objects. In the example below, we can see the IP address and port for the remote create instance response:



The resolution process is anonymous, so we need to understand where the victim is being redirected. This requires intercepting the EPMAPPER or OXID resolution process to determine the destination endpoint's port.

Once we have this information, we set up a listener for the designated port, wait for incoming connections, extract the ticket for our relay, and transparently forward the victim's request to the actual destination and service.

We mentioned previoulsy about the **Muthual Autentication**. This flag is set by the client in the "ap-options":



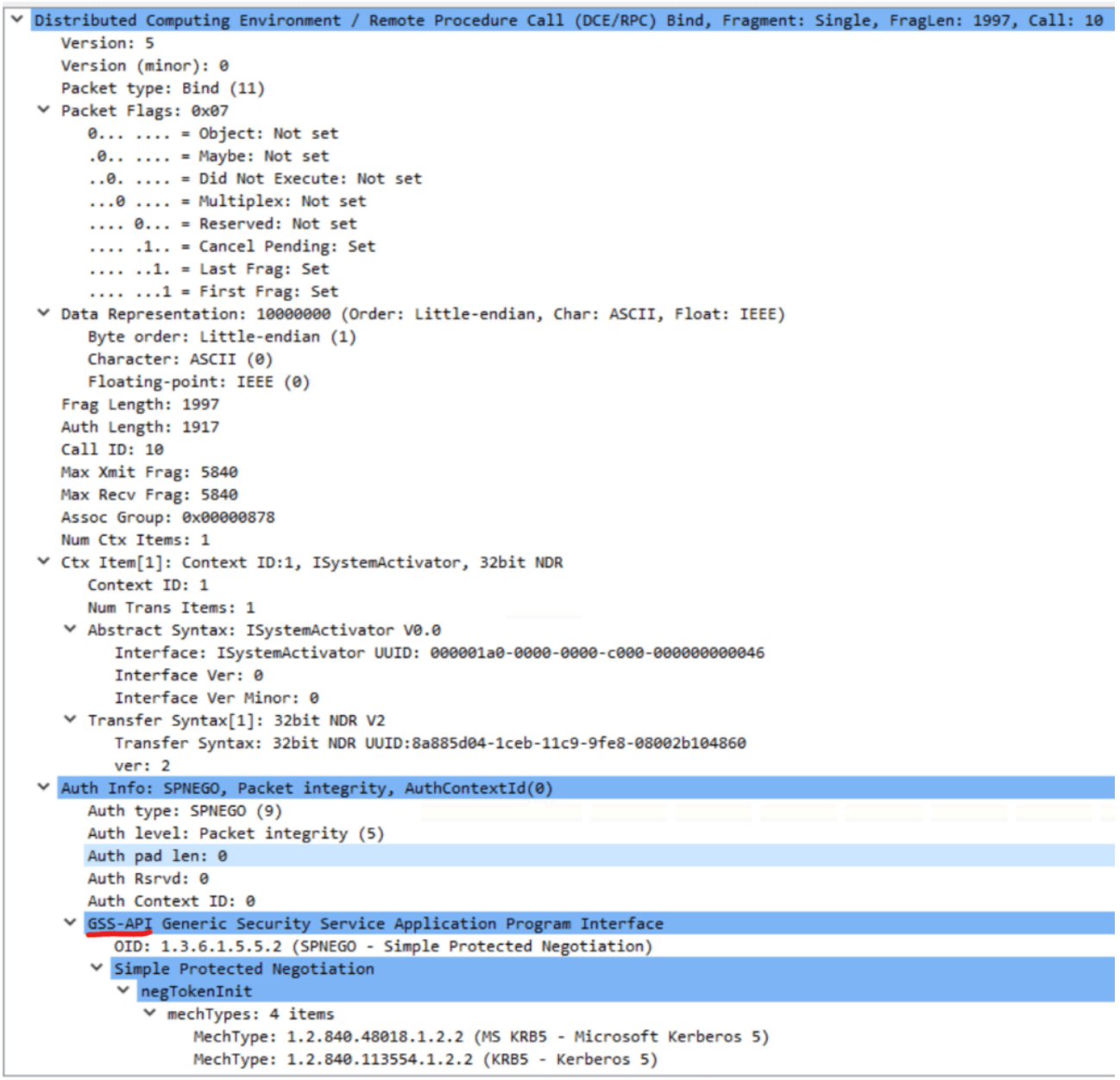
If the client sets the **ISC_REQ_MUTUAL_AUTH** flag when generating the AP_REQ, it expects an **AP_REP** from the server to confirm mutual authentication. The AP_REP proves the server holds the correct key. If it's missing or invalid, the client may reject the connection, assuming the server is untrusted.

In a relay scenario, mutual authentication isn't a usually a concern ,remember, the server is the target, not the client. Once the server accepts the AP_REQ, the attacker has succeeded. The AP_REP sent back can be ignored.

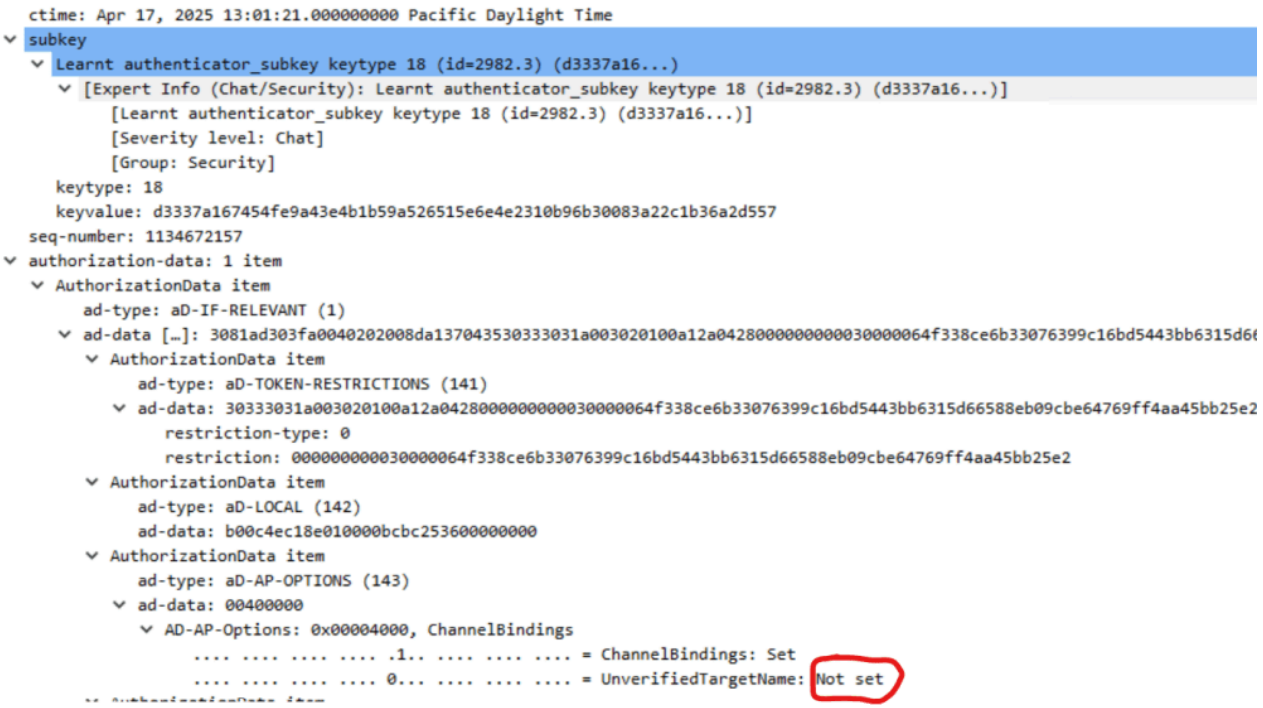
However, in some cases, this step cannot be skipped. The server may respond with "**more processing required**", indicating that an AP_REP must be returned to the client. In this case, we need to extract the AP_REP, encapsulate it within the victim's protocol, send it back, and wait for the client to send a new, validated AP_REP

This consistently happened when I was relaying RPC/DCOM instead of SMB, so I got curious and wanted to understand why.

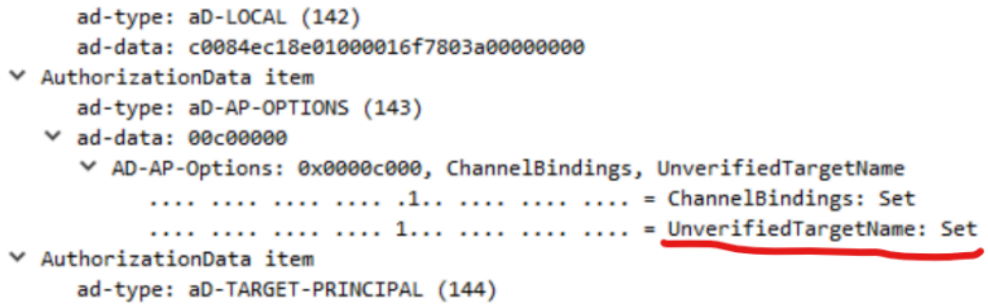
At first, I assumed this behavior was caused by the three-leg DCE-style mutual authentication. However, after reviewing the specifications, I realized that this three way handshake only occurs when GSS-API wrapping is not used ,which doesn't apply here, since our originating packet is clearly encapsulated in GSS-API.



After spending some time in Wireshark decrypting the tickets, I noticed that when relaying to SMB, the AP-REQ does not have the **UnverifiedTargetName** flag set:



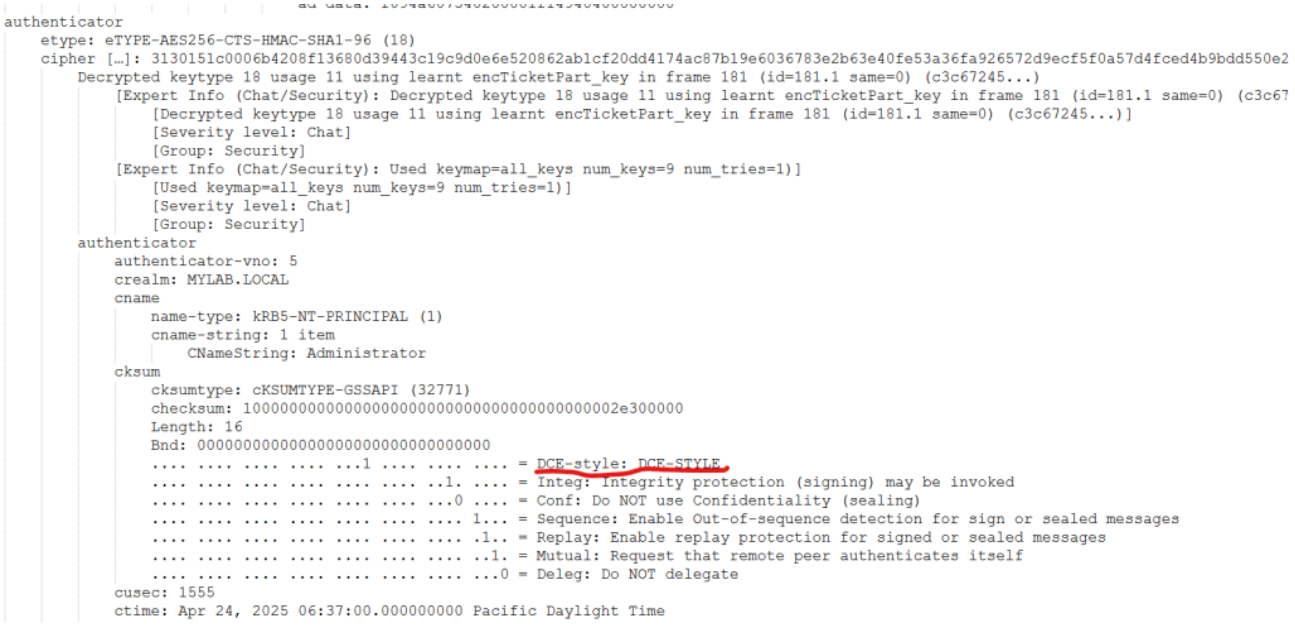
But with RPC/DCOM the flag will be set:



The **ISC_REQ_UNVERIFIED_TARGET_NAME** flag was introduced by Microsoft to protect from SPN generated by untrusted sources.

So I thought: *since both protocols set Mutual Authentication to true, maybe the* ***UnverifiedTargetName*** *flag signals that the server also expects a confirmation from the client?*

But a deeper analysis in Wireshark of the decrypted Authenticator revealed that DCE-STYLE was also set when using GSS-API.



Neverthless, this doesn't mean authentication fails, it just requires an extra round trip

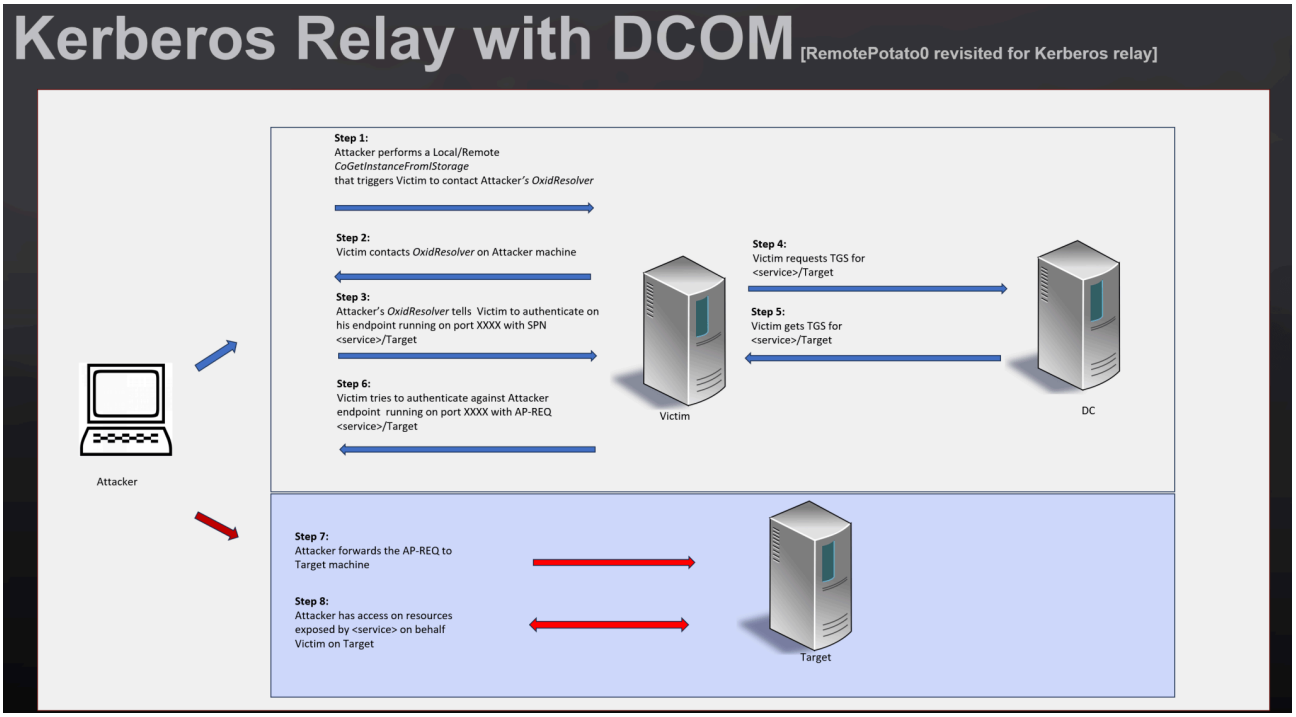
spOofing spn

In this scenario, the attacker is able to coerce,either locally or remotely, a victim into authenticating to an endpoint under the attacker's control using a forged SPN.

This technique can be leveraged for DCOM relaying, something Antonio Cocomazzi (aka @splinter_code) and I previously demonstrated using NTLM with our RemotePotato0 tool, by abusing the classic *Potato techniques

Later, James Forshaw published a great article showing how the same concept can be applied to relay DCOM over Kerberos instead of NTLM.

Without going too deep, the diagram below outlines the process clearly. The key takeaway is that in our fake OxidResolver response, we can specify **any SPN we want** and **direct the victim where to connect**. The victim will then request a TGS for that SPN and send the AP-REQ to our controlled endpoint.



This kind of attack can be done with the KrbRelay framework or my KrbRelayEx-RPC tool.

There are some limitations, notably, several CLSIDs used for instantiating DCOM objects will set an Identify-level token, which cannot be used for relay and results in a **BAD_IMPERSONATION_LEVEL** error.

```
        => 'l' for listing connected clients
[*] FakeRPCServer[135]: Client connected [192.168.212.23:53006] in RELAY mode
[*] FakeRPCServer[135]: Client connected [192.168.212.23:53007] in RELAY mode
[*] SMB relay [192.168.212.23:53007] Connected to: PKI-MYLAB.MYLAB.LOCAL:445
[*] SmbClient [192.168.212.23:53007] STATUS_MORE_PROCESSING_REQUIRED
[*] Header Status STATUS_SUCCESS
[*] SMB Login  success: True
[-] Could not connect to ipc$ : STATUS_BAD_IMPERSONATION_LEVEL
[*] Exiting from  ipc$
```

Indeed, there is a first authentication, shown as **Step 2** in the diagram above, that will always use a valid impersonation token. However, the SPN in this case is not under our control and will always default to something like RPCSS/<ip_address>.

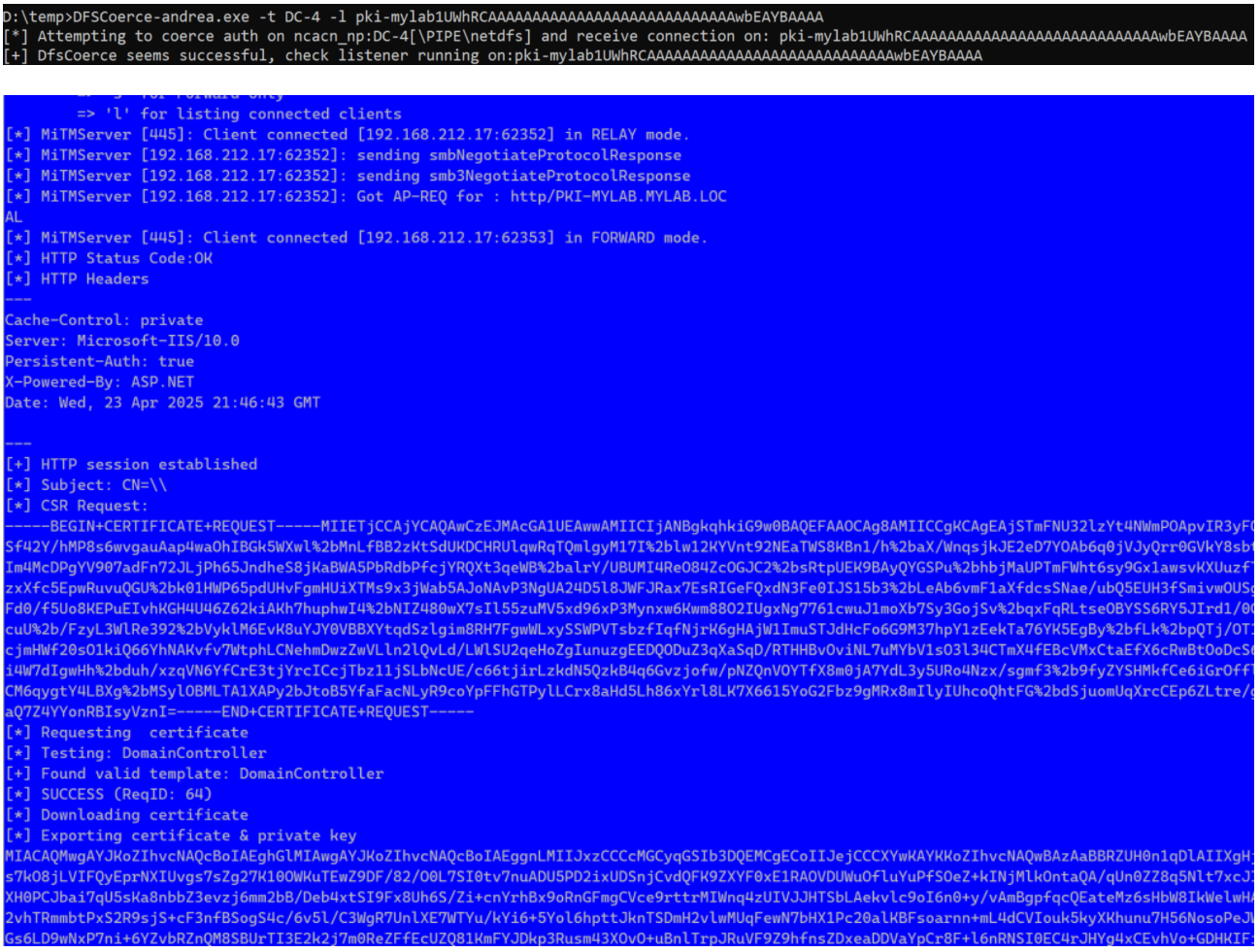
But there's a clever workaround. Using James Forshaw's Marshaled Target Information SPN trick, you can bypass this limitation by registering a specially crafted DNS hostname in the form:
<HOSTNAME>1UWhRCAAAAAAAAAAAAAAAAAAAAAAAAAAAwbEAYBAAAA

The victim will request a TGS for **RPCSS/<HOSTNAME>**, but will actually connect to the IP address of the forged hostname.

Since in many environments a regular domain user can register DNS records, this limitation can be bypassed

This “trick” is also applicable to other scenarios and protocols, like SMB. For example, we can use the classic PetitPotam or, even better, DFSCoerce and DFSCoerce-2, to coerce authentication from a computer or domain controller account and relay it to our target service.

In the following screenshots, a domain user coerces a Domain Controller to authenticate to our KrbRelayEx tool using the SMB protocol via DFSCoerce. We then exploit the ESC8 vulnerability to relay this authentication to the ADCS Web Enrollment service and obtain a client authentication certificate, which can be used to impersonate the Domain Controller.



CONCLUSIons

By the end of this post, I hope Kerberos relay attacks are clearer and that it’s evident Kerberos itself doesn’t inherently prevent relay attacks.

As Elad Shamir wrote “Kerberos is **NOT** the Solution” in this case.

The most effective way to mitigate relay attacks is by enforcing both signing and channel binding on all servers. If this isn’t already in place, **it should be implemented as a priority**, rather than spending time chasing complex detections for these attacks.

That’s all